

Homework 4: Spectre Attack Report

Hsing-Yu Lin

June 6, 2026

Abstract

This report describes my implementation of a Spectre-style cache side-channel attack against a bounds-checked victim function. The attack trains the branch predictor with repeated legal calls, delays the resolution of the bounds check by flushing `array1_size`, and recovers the secret byte by observing which cache line of `array2` becomes fast to reload after transient execution. The implementation uses repeated trials, branchless payload selection, shuffled reload order, timestamp-based cache probing, threshold calibration or manual threshold tuning, and filtering of the legal training value. In the provided runs, the attack recovered the 40-byte secret with one run reaching 39/40 confident correct bytes and another run reaching 40/40 confident correct bytes.

1 Victim and Attack Goal

The victim function performs an architectural bounds check before accessing `array1`:

```
1 void victim_function(size_t x)
2 {
3     if (x < array1_size) {
4         temp ^= array2[array1[x] * CACHE_LINE_SPACING];
5     }
6 }
```

Architecturally, an out-of-bounds value of `x` should not access `array1[x]`. However, if the branch predictor has been trained to expect the bounds check to be true, the processor may transiently execute the branch body for a malicious out-of-bounds `x`. Although the speculative execution is eventually squashed, the access to `array2[array1[x] * 512]` can leave a measurable cache footprint. The attack therefore converts a secret byte into a cache state and then recovers it through timing measurements.

2 Local Branch Predictor Training

Each attack repetition uses five legal calls followed by one malicious call:

```
1 for (int j = 5; j >= 0; j--) {
2     flush_and_wait(&array1_size);
3     for (volatile int z = 0; z < 100; z++) { }
4
5     size_t mask = ((j % 6) - 1) & ~0xFFFF;
6     mask = mask | (mask >> 16);
7     size_t x = training_x ^ (mask & (malicious_x ^ training_x));
8
9     victim_function(x);
10 }
```

For $j = 5, 4, 3, 2, 1$, the selected x is a legal in-bounds index. These calls train the predictor to expect the victim branch $x < \text{array1_size}$ to be taken. For $j = 0$, the selected x is the malicious out-of-bounds index. If the predictor still predicts the branch as taken, the victim transiently accesses `array2` using the secret byte as the index.

3 Global Branch-History Stabilization and Branchless Selection

The assignment notes that the victim branch may be predicted using not only local history but also global branch history. Since the exact predictor structure of the tested processor is unknown, my implementation does not attempt to model the predictor precisely. Instead, it tries to make the control-flow pattern before each victim call as repeatable as possible.

The most important choice is to avoid an additional if or ternary operator when selecting between the legal training index and the malicious index. This prevents an extra branch from modifying the global branch history immediately before the victim bounds check.

The mask expression is intentionally kept in the code:

```
1 mask = ((j % 6) - 1) & ~0xFFFF;
2 mask = mask | (mask >> 16);
3 x = training_x ^ (mask & (malicious_x ^ training_x));
```

This computes the same logical choice as:

```
1 if (j == 0)
2     x = malicious_x;
3 else
4     x = training_x;
```

but without introducing an additional branch in the critical attack path.

Important observation during debugging. I tested an if-based version of the payload selection and observed that the result dropped to 0/40 recovered bytes in the tested environment. This does not necessarily mean that the if version selected the wrong value of x . A more likely explanation is that the extra if branch changed the branch predictor state or global branch history before the call to `victim_function`. As a result, the processor no longer reliably mispredicted the victim bounds check during the malicious call. Therefore, keeping the branchless mask-based selection is not only a style choice; it is an important robustness technique for preserving a stable predictor history.

I also execute a short deterministic loop before the victim call. This loop is not the main mechanism for extending the transient window; that role is played by flushing `array1_size`. Rather, the loop helps create a repeatable branch pattern before the victim call, which empirically improved stability on the tested machine.

```
1 for (volatile int z = 0; z < 100; z++) { }
```

4 Extending the Speculation Window

Before each victim call, the code flushes `array1_size`:

```
1 flush_and_wait(&array1_size);
```

The branch condition depends on `array1_size`. If this variable is not cached, the processor needs more time to resolve the comparison $x < \text{array1_size}$. This gives transient execution more time to execute the dependent memory access inside the branch body and to bring `array2[secret * 512]` into the cache.

5 Cache Side Channel and Scoring

At the beginning of each repetition, all candidate lines in `array2` are flushed:

```
1 for (int i = 0; i < 256; i++) {  
2     __mm_clflush(&array2[i * CACHE_LINE_SPACING]);  
3 }  
4 __mm_mfence();
```

After the training and malicious calls, the attack reloads the 256 candidate lines in a shuffled order:

```
1 int mix_i = ((i * 167) + 13) & 255;  
2 addr = &array2[mix_i * CACHE_LINE_SPACING];
```

The shuffled probing order avoids a simple sequential traversal that could create hardware prefetcher noise. A reload time below the chosen threshold is counted as a cache hit. The implementation keeps a `hit_counts[256]` table and returns the two byte values with the largest hit counts as the best and second-best guesses.

The timing measurement uses `__rdtscp` and `__mm_lfence` to reduce timing reordering around the measured load. The implementation can either use a manually supplied threshold or run a calibration routine that compares cached and flushed access-time distributions.

6 Other Accuracy Techniques

The attack also uses the following robustness techniques:

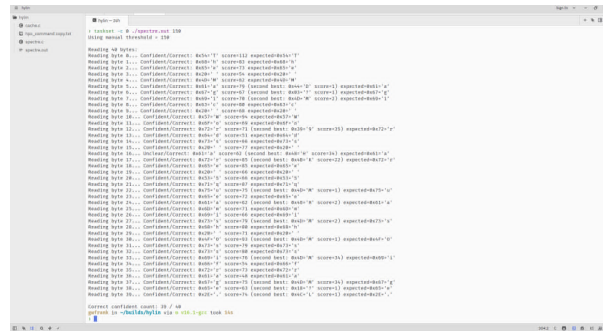
1. **Repeated trials.** Each byte is attacked over `NUM_TRIES = 10000` repetitions, so the output is based on statistical voting.
2. **Cache-line spacing.** `CACHE_LINE_SPACING = 512` separates candidate values in `array2`, reducing accidental overlap between candidate cache lines.
3. **Training-value filtering.** The line loaded by the legal training calls is ignored during scoring.
4. **Serialized timing.** `__rdtscp` and `__mm_lfence` are used around the timed load to reduce measurement noise.
5. **Manual and calibrated thresholds.** The code supports both automatic calibration and manual threshold selection, which is useful because cache-hit thresholds are machine-dependent.

7 Experimental Results

The secret string used in the experiment was:

The Magic Words are Squeamish Ossifrage.

Two representative manual-threshold runs were observed. The first run recovered 39 out of 40 bytes as both confident and correct (With a manual reload threshold of 150 cycles). The second run recovered 40 out of 40 bytes as both confident and correct. (With a manual reload threshold of 163 cycles)



The implementation successfully demonstrates how speculative execution can bypass an architectural bounds check and encode a secret-dependent value into the cache state. The most important techniques are local predictor training through legal victim calls, branchless selection to avoid disturbing global branch history, flushing `array1_size` to extend the speculation window, repeated Flush+Reload measurements, shuffled probing, and filtering of the legal training cache line. The best observed run recovered 40/40 bytes confidently and correctly.